OFFSIDE LABS

Dynamic Vault

Smart Contract Security Assessment

November 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Siji Feng

Ronny Xing

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Meteora Dynamic Vault* smart contracts, starting on November 5th, 2025, and concluding on November 10th, 2025.

Project Overview

PR297 adds support for Jupiter Lend as a new yield strategy and updates Kamino integration to use their latest program version, while removing deprecated Marginfi support.

Audit Scope

The assessment scope contains mainly the smart contracts of the vault program for the *Meteora Dynamic Vault* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Mercurial Vault
  - Codebase: `https://github.com/MeteoraAg/mercurial-vault`
  - Commit Hash: cc626c9503ebfe8cc92a782c4c5c75388ec81523

We listed the files we have audited below:

- Mercurial Vault
  - programs/vault/src/*.rs

Findings

The security audit revealed:

- 0 critical issue
- 1 high issue
- 0 medium issue
- 0 low issue
- 0 informational issue

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Unvalidated Obligation Account in Kamino Strategy	High	Fixed

4 Key Findings and Recommendations

4.1 Unvalidated Obligation Account in Kamino Strategy

Severity: High	Status: Fixed
Target: Smart Contract	Category: Missing Validation

Description

The `get_collateral_amount` helper function in the Kamino strategy reads collateral amounts from a Kamino obligation account without validating the its association with the vault's strategy. This allows a malicious operator to pass an arbitrary (including empty) obligation account, corrupting the vault's accounting system.

```
21    fn get_collateral_amount<'info>(
22        reserve_key: Pubkey,
23        obligation: &'info AccountInfo<'info>,
24    ) -> Result<u64> {
25        let loader: AccountLoader<Obligation> =
          ↪ AccountLoader::try_from(obligation)?;
26        let obligation_state = loader.load()?;
27
28        if let Ok(obligation_collateral) =
          ↪ obligation_state.find_collateral_in_deposits(reserve_key) {
29            return Ok(obligation_collateral.deposited_amount);
30        }
31        return Ok(0);
32    }
```

programs/vault/src/strategy/kamino.rs#L21-L32

The function only checks that `obligation` is a valid `Obligation` account, but does not verify that it belongs to the strategy. It relies on subsequent CPI calls to Kamino for validation, but these can be bypassed through early exit logic.

1. `remove_strategy2` (Admin only): No CPI validation is performed after reading collateral amount, allowing accounting corruption.
2. `handler_withdraw_strategy` (Operator only): Kamino's `withdraw` function contains early exit logic that can bypass CPI validation.

```rust
    let deposited_amount =
        get_collateral_amount(reserve.key(),
↪ kamino_accounts.obligation.account_info())?;
    if deposited_amount == 0 {
        return Ok(());
    }
    // always keep at least 1 in obligation to avoid the obligation
    ↪ account is closed
    let amount = amount.min(deposited_amount.checked_sub(1).unwrap());
    if amount == 0 {
        return Ok(());
    }
```

programs/vault/src/strategy/kamino.rs#L312-L321

3. `handler_withdraw_directly_from_strategy` (Permissionless): An incorrect `Obligation` status misleads the strategy into believing its liquidity is (almost) zero, which could result in a huge loss. We have, however, ensured that the maximum `precision_loss` is 1, thereby preventing state corruption.

```rust
    require!(precision_loss <= 1, VaultError::InvalidPrecisionLoss);
```

programs/vault/src/instructions/deposit_withdraw_liquidity.rs#L336-L336

Impact

A malicious operator can manipulate vault accounting to show false losses or gains, thereby corrupting the records and draining the vault.

Proof of Concept

1. Malicious operator creates an empty Kamino obligation and calls `withdraw_strategy` with this wrong obligation. The vault reads 0 collateral, records a massive loss, and deflates `total_amount` .
2. Any user deposits tokens when `total_amount` is deflated. Due to the deflated accounting, they receive a large share of LP tokens.
3. Operator performs another rebalance (e.g., deposit) using the correct obligation. The vault "rediscovers" the original funds, calculates a huge "fake gain" and inflates `total_amount` .
4. After waiting hours for locked profit to be unlocked, the user withdraws their LP tokens. They could drain a massive amount of funds from the vault.

Recommendation

- The `get_collateral_amount` trait has the `strategy` Pubkey in the arguments. We can verify that the `owner` of the `Obligation` matches the associated `strategy` .
- Remove the early-exit cases in withdrawals, as they do not provide any real benefit.

- Add more precision loss checks. Limiting the maximum allowable loss helps mitigate the risk of a sudden drop in strategy liquidity.

Mitigation Review Log

Fixed in PR301, commit 57189ff1a6ced68d3e587ea6620d9504a47a668e.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

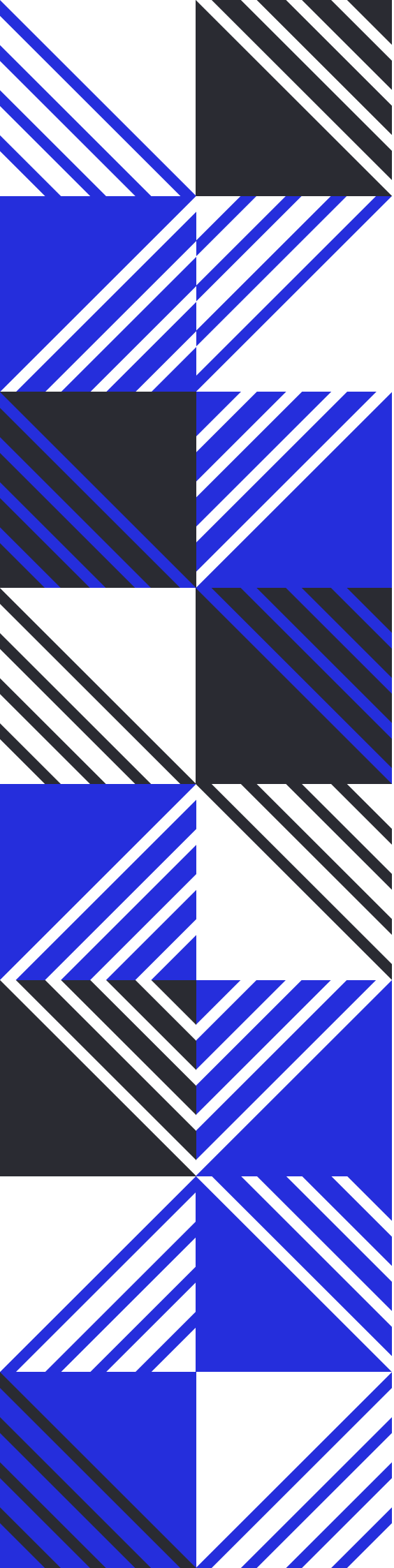
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs